VOICE-FIRST

Frontend Development Master Prompt Pack

Voice-First Paid Calling Platform - React Native Mobile App + React Admin Website

Production-oriented frontend implementation sequence - July 2026 baseline

52 sequential, copy-paste-ready prompts covering architecture, design, motion, haptics, customer flows, host flows, admin frontend, testing, performance, CI/CD, store builds, OTA updates, and final release audit.

Read this before using the prompts

What this pack delivers. A production-oriented frontend monorepo containing an Expo/React Native customer-and-host mobile app and a Next.js admin website. It includes mock adapters and typed contracts so every screen and flow can be developed and deployed before the backend is ready.

Important limitation. A frontend alone cannot provide secure OTP authentication, real wallet balances, verified Razorpay payments, Agora access tokens, fraud decisions, refunds, or host payouts. The prompts therefore create production-quality UI, state machines, adapters, mocks, environment configuration, and integration boundaries. Live commercial operation still requires backend APIs and provider credentials later.

How to execute. Run the prompts in order against the same repository. Give the coding agent the current repository each time. Do not skip acceptance criteria. Commit after each major phase. When an external asset or credential is unavailable, the agent must implement a typed fallback and document the exact external prerequisite instead of leaving the project broken.

Technology baseline

Area	Selected technology	Primary responsibility
Mobile framework	Expo + React Native + TypeScript	Android/iOS app, native modules, development builds
Mobile routing	Expo Router	Typed file-based routes, deep links, route groups
Mobile styling	NativeWind + shared semantic tokens	Layouts, spacing, typography, themes; StyleSheet for dynamic drawing
Motion	Reanimated + Gesture Handler	Micro-interactions, layout motion, gestures, shared values
Illustrative animation	Lottie	Success, failure, empty, approval, and celebration sequences
Interactive animation	Rive	Signature conversation orb and state-machine visuals
Custom graphics	React Native Skia	Trust ring, waveform, particles, visual meters
Haptics	Expo Haptics	Selection, success, warning, rejection, call events
Lists	FlashList	Hosts, history, leaderboard, transactions, notifications
State	Zustand + TanStack Query	Client state and server-like cache/mocks
Forms	React Hook Form + Zod	Typed validation and accessible errors
Admin	Next.js App Router + Tailwind + shadcn/ui	Six-module responsive operations website
Testing	Jest + RNTL + Playwright + Maestro	Unit, component, web E2E, mobile E2E
Monitoring	Sentry	Errors, traces, source maps, release diagnostics
Delivery	EAS Build/Submit/Update + web hosting	Mobile binaries, store submission, OTA UI fixes, admin deployment

Locked visual and interaction direction

- Brand: premium, friendly, trustworthy, energetic, and safe - never suggestive or anonymous.
- Primary palette: indigo/purple; accent: coral; call surfaces: deep navy gradients; financial success: green; warnings: amber; destructive actions: red.
- Typography: Inter with a compact mobile type scale and tabular numerals for timers, wallet balances, earnings, and ranks.
- Spacing: 4-point grid; mobile horizontal gutter 16; large screens 20-24; minimum touch target 44x44; rounded surfaces 12/16/24.
- Motion: 120 ms micro feedback, 220 ms component transitions, 300 ms screen transitions, 340 ms sheets, 800-1600 ms one-time celebrations.

- Accessibility: visible focus, semantic labels, dynamic type tolerance, reduced motion, colour-independent statuses, and WCAG-conscious admin contrast.

Prompt map

Section	Prompts
Foundation and architecture	1-12
Customer mobile experience	13-24
Host mobile experience	25-34
Admin website	35-40
Cross-cutting production quality	41-52

Foundation and architecture

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were completed and committed.

Prompt 01 - Repository audit and implementation plan

Outcome: Audit the current repository and produce an actionable frontend build plan without deleting working code.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Audit the current repository and produce an actionable frontend build plan without deleting working
code.

IMPLEMENTATION REQUIREMENTS
1. Inspect all files, package manifests, current Figma-derived screens, routes, components, assets,
tests, lint rules, and build configuration.
2. Determine whether the repository is empty, a single app, or already a monorepo. Identify code that
can be retained and code that must be migrated.
3. Create `docs/frontend-audit.md` with:
   - current structure and dependency inventory;
   - route/screen coverage against the approved customer, host, and six-module admin flows;
   - duplicate or inconsistent UI patterns;
   - native-module and Expo development-build requirements;
   - risks on Windows, Android, iOS, and web;
   - phased implementation order mapped to Prompts 2-52.
4. Create `docs/screen-inventory.md` listing every screen, role, route, primary actions, states,
motion requirement, mock data dependency, and completion status.
5. Create `docs/decision-log.md` and record the chosen monorepo, design-token, state-management,
animation, testing, and deployment decisions.
6. Do not perform a broad rewrite in this stage. Only make safe audit-enabling fixes such as missing
scripts or broken configuration that prevents inspection.

Acceptance criteria:
- The audit names every existing route and identifies missing routes.
- The plan distinguishes mobile app, admin website, shared packages, and external backend
prerequisites.
- No existing working screen or prototype flow is removed.
- The repository passes its pre-existing checks, or every pre-existing failure is documented with
evidence.
```

Prompt 02 - Production monorepo bootstrap

Outcome: Create a clean pnpm monorepo that can host the Expo mobile app, Next.js admin website, and shared packages.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Create a clean pnpm monorepo that can host the Expo mobile app, Next.js admin website, and shared packages.

IMPLEMENTATION REQUIREMENTS
Create or migrate to this structure while preserving useful existing work:
- `apps/mobile` - Expo Router React Native app for customer and host roles.
- `apps/admin` - Next.js App Router admin website.
- `packages/design-tokens` - platform-neutral colour, spacing, typography, radius, shadow, z-index, and motion values.
- `packages/domain` - frontend domain types, Zod schemas, enums, state-machine events, mock contracts.
- `packages/eslint-config`, `packages/tsconfig`, and optional `packages/test-utils`.
- `docs` - architecture, dependency matrix, screen inventory, release instructions.

Configure:
- root `pnpm-workspace.yaml`, scripts, `.editorconfig`, `.gitignore`, `.npmrc`, Node engine, package manager field, and workspace aliases;
- TypeScript project references or an equivalent reliable workspace setup;
- strict imports so app code does not reach into another app's internal folders;
- consistent build, dev, clean, lint, typecheck, test, and format commands from the root;
- Expo development-build support, not an Expo Go-only workflow;
- Next.js production build and start scripts;
- shared environment naming conventions without committing secrets.

Do not couple mobile UI directly to admin UI. Share semantic tokens and domain contracts, not platform-specific components.

Acceptance criteria:
- `pnpm install` succeeds from a clean checkout.
- Root commands can lint, typecheck, test, and build each workspace independently.
- Both apps boot to a simple branded health screen.
- Workspace dependency boundaries are documented and enforced.
```

Prompt 03 - Quality tooling and commit gates

Outcome: Install production-grade formatting, linting, static analysis, and pre-commit quality gates.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.
```

```
Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Install production-grade formatting, linting, static analysis, and pre-commit quality gates.

IMPLEMENTATION REQUIREMENTS
Implement:
- ESLint flat configuration suitable for Expo/React Native, Next.js, TypeScript, React Hooks, Testing
Library, and import boundaries.
- Prettier with Tailwind/NativeWind class ordering where applicable.
- TypeScript strict mode, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes` where compatible,
and aliases.
- Husky or an equivalent maintained git-hook approach with lint-staged.
- commit-safe scripts that do not modify unrelated files.
- dead-code and dependency checks using a maintained tool; configure explicit ignores for Expo/Next
generated files.
- spell-check or copy lint for user-facing text where practical.
- CI-friendly `check` command that runs formatting verification, lint, typecheck, unit tests, and
production builds.

Add `CONTRIBUTING.md` describing branch naming, commit expectations, quality commands, and how to add
a screen, route, animation, or mock endpoint.

Acceptance criteria:
- Deliberate lint, formatting, and type errors are caught.
- Hooks run quickly on staged files; full checks remain available for CI.
- No blanket `eslint-disable`, `ts-ignore`, or `any` is introduced to make checks pass.
```

Prompt 04 - Environment, runtime configuration, and feature flags

Outcome: Create validated frontend runtime configuration for local mocks, staging APIs, production APIs, animations, analytics, and provider selection.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
```

```
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Create validated frontend runtime configuration for local mocks, staging APIs, production APIs,
animations, analytics, and provider selection.

IMPLEMENTATION REQUIREMENTS
Create typed configuration layers for both apps.

Mobile configuration must include public values such as app environment, API base URL placeholder,
Agora app ID placeholder, Razorpay key ID placeholder, Sentry DSN placeholder, mock mode, animation
feature flags, and deep-link host. Never expose server secrets, Razorpay secret, Agora certificate,
signing keys, or private tokens.

Admin configuration must include API base URL placeholder, Sentry DSN, deployment environment, mock
mode, and feature flags.

Requirements:
- validate variables with Zod at startup/build time;
- provide `.env.example` files with comments;
- implement `development`, `preview`, and `production` profiles;
- create a typed `featureFlags` module supporting graceful disabling of Rive, Skia-heavy visuals,
video upgrade, referrals, and analytics;
- show a non-production environment badge in development and preview builds only;
- fail clearly when a required production-safe public value is missing;
- document which future backend secrets must never enter the frontend.

Acceptance criteria:
- Invalid production configuration fails before release.
- Mock mode can run every frontend flow without network access.
- No secret-looking values are committed or printed to logs.
```

Prompt 05 - Shared domain model and state machines

Outcome: Define the complete frontend domain model and explicit state machines before implementing screens.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Define the complete frontend domain model and explicit state machines before implementing screens.
```

```
IMPLEMENTATION REQUIREMENTS
In `packages/domain`, define strongly typed models and Zod schemas for:
- roles, customer, host, admin summary, language, host rank, host availability;
- wallet, payment, transaction, referral, earnings, withdrawal, refund, report, support ticket,
notification;
- call session, media mode, participant, pricing snapshot, timer snapshot, connection quality, call
end reason;
- host application and verification status.

Create pure state machines or reducer-style transition tables for:
1. app bootstrap/auth routing;
2. outgoing call: idle -> permission check -> balance check -> ringing -> accepted/rejected/no-
answer/busy -> connected;
3. active call: voice locked -> voice eligible -> video requested -> accepted/rejected -> video ->
voice -> reconnecting -> ended;
4. payment: idle -> order requested -> checkout -> verification pending -> success/failure/pending;
5. host application;
6. withdrawal;
7. referral qualification.

Every transition must reject invalid events and have unit tests. UI screens must later consume these
states rather than inventing booleans.

Acceptance criteria:
- There is one canonical source for enums and state transitions.
- Pricing and rank snapshots are immutable within a completed call record.
- Invalid transitions are test-covered and cannot silently occur.
```

Prompt 06 - Shared visual design tokens

Outcome: Implement the shared production design-token package and platform mappings.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the shared production design-token package and platform mappings.

IMPLEMENTATION REQUIREMENTS
Create semantic tokens rather than using raw colours throughout screens.

Required colour roles:
- brand primary, primary pressed, primary soft;
- coral accent and accent soft;
- background, elevated background, surface, surface muted;
- text primary, secondary, tertiary, inverse;
- border subtle/strong, focus ring;
```

```
- success, warning, error, info and their soft surfaces;
- call background start/end, call control surface, video overlay;
- rank bronze/silver/gold/platinum/diamond with readable foregrounds.

Create:
- 4-point spacing scale from 0 to 64;
- mobile gutters 16/20/24;
- radii 8/12/16/20/24/full;
- type scales and line heights for mobile and admin;
- tabular number style guidance;
- elevation/shadow recipes with Android and iOS mappings;
- z-index layers;
- breakpoints for admin and tablet-safe mobile layouts;
- motion durations and easing names;
- opacity and disabled-state rules.

Export tokens as TypeScript and CSS variables. Add a visual token gallery route in both apps.

Acceptance criteria:
- No component needs to import hard-coded brand colours.
- Light and dark mobile themes are supported, while admin defaults to a polished light theme with
optional dark mode.
- Contrast checks are documented for critical text and statuses.
```

Prompt 07 - Mobile routing and role architecture

Outcome: Implement Expo Router route groups, guards, deep links, and role-specific navigation.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement Expo Router route groups, guards, deep links, and role-specific navigation.

IMPLEMENTATION REQUIREMENTS
Create a typed route structure for:
- bootstrap and mandatory-update handling;
- `(auth)` age confirmation, login, OTP, referral code;
- `(customer-onboarding)` profile, language, permissions;
- `(customer)` tab routes: home, calls, wallet, profile;
- customer modal/sheet routes: filters, add money, payment states, referrals, support, refund;
- `(host-onboarding)` host profile, photo, rules, verification, payout, agreement, application states;
- `(host)` tabs: dashboard, calls, earnings, profile;
- shared `call` stack: outgoing, voice, video request, video, reconnecting, report, summary;
- legal and account-status routes.

Implement:
```

```
- auth and role guards using the store, not component-level redirects scattered everywhere;
- restore-last-safe-route behaviour;
- deep links for referrals, support ticket, payment result, notification destination, and call
invitation placeholder;
- not-found and invalid-link screens;
- route analytics hook stub;
- no double navigation on rapid taps.

Acceptance criteria:
- A customer cannot open host-only routes and vice versa.
- Logged-out deep links preserve the intended destination through authentication when safe.
- Every route is typed and has a deterministic back-navigation policy.
```

Prompt 08 - Mobile styling and responsive layout foundation

Outcome: Configure NativeWind and React Native layout foundations without sacrificing dynamic animation performance.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Configure NativeWind and React Native layout foundations without sacrificing dynamic animation
performance.

IMPLEMENTATION REQUIREMENTS
Install and configure the stable NativeWind version compatible with the selected Expo SDK. Do not use
a pre-release unless the compatibility audit explicitly justifies it.

Implement:
- token-driven Tailwind/NativeWind theme mappings;
- global font loading for Inter weights actually used;
- safe-area-aware `Screen`, `ScrollScreen`, `KeyboardScreen`, and `CallScreen` primitives;
- consistent status bar/navigation bar handling;
- responsive helpers for compact phones, standard phones, large phones, and tablets;
- keyboard-safe forms and bottom actions;
- RTL-aware spacing primitives even if RTL languages are not yet enabled;
- StyleSheet/Reanimated styles for highly dynamic properties; do not force every animation through
utility classes.

Create a layout QA screen that demonstrates gutters, grids, text wrapping, long names, large fonts,
landscape call layout, keyboard overlap, and small Android devices.

Acceptance criteria:
- Screens contain no arbitrary one-off margins when a token exists.
- Primary bottom actions remain reachable with keyboard and safe-area insets.
- Text does not clip at 200% font scaling on core auth and payment screens.
```

Prompt 09 - Reusable mobile component system

Outcome: Build the complete reusable mobile UI component library before feature screens.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Build the complete reusable mobile UI component library before feature screens.

IMPLEMENTATION REQUIREMENTS
Create accessible, tokenized, documented components with variants:
- Button: primary, secondary, tonal, ghost, destructive, icon-only, compact, loading, disabled.
- IconButton and CallControlButton with pressed scale and haptic policy.
- TextField, PhoneField, OTPInput, SearchField, SelectField, TextArea, Checkbox, RadioCard, SwitchRow.
- AppHeader, BackHeader, WalletHeader, TabBar, SectionHeader.
- Card, HostCard shell, MetricCard, WalletCard, RankBadge, StatusBadge, Avatar, EmptyState,
ErrorState, Skeleton.
- Banner, Toast, Snackbar, InlineError, ConfirmationDialog.
- BottomSheet wrapper using Gorhom Bottom Sheet.
- ListRow, TransactionRow, CallHistoryRow, ReferralRow, NotificationRow.
- PermissionExplainer and LegalContent screen shell.

Use Lucide icons through explicit imports to protect bundle size. Add accessibility labels, roles,
hints, hit slop, and test IDs. Add a component-gallery route with every state.

Acceptance criteria:
- Components have no feature-specific data fetching.
- Visual states match the token gallery.
- Press feedback works with reduced motion and haptic preferences.
- Critical components have React Native Testing Library coverage.
```

Prompt 10 - Motion and haptics infrastructure

Outcome: Create a central motion and haptics system used consistently throughout the app.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
```

```
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Create a central motion and haptics system used consistently throughout the app.

IMPLEMENTATION REQUIREMENTS
Implement:
- `MotionProvider` reading system Reduce Motion and app preferences;
- motion tokens: instant 100-120 ms, control 160-180 ms, component 220-250 ms, screen 280-320 ms,
sheet 320-380 ms, celebration 900-1500 ms;
- approved easings and spring presets: gentle, responsive, expressive, no-bounce;
- reusable Reanimated helpers for fade/slide/scale, list stagger, shared element substitute, number
transitions, skeleton shimmer, pulse, and layout changes;
- gesture utilities with Gesture Handler;
- `HapticsService` mapping selection, light impact, medium confirm, warning, reject, success, call
accepted, video unlocked, payment result, low balance, and rank-up events;
- a user preference to disable nonessential haptics;
- no haptics during camera use when the platform does not deliver them reliably;
- prevention of repeated haptics during rerenders.

Create a motion-lab route demonstrating every transition and reduced-motion fallback.

Acceptance criteria:
- Feature code calls semantic events such as `haptics.videoUnlocked()` instead of raw platform APIs.
- Reduced motion removes large movement and repeated pulses but preserves state clarity.
- Animations are cancelled safely on navigation and unmount.
```

Prompt 11 - Lottie, Rive, and Skia visualization framework

Outcome: Integrate each animation technology for a specific purpose with robust fallbacks.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.
```

```
STAGE OBJECTIVE
Integrate each animation technology for a specific purpose with robust fallbacks.

IMPLEMENTATION REQUIREMENTS
Implement an animation asset registry and wrappers:
- Lottie wrapper for one-time or looped illustrative JSON/.lottie assets, with play, pause, reduced-
motion static frame, error fallback, recolouring where supported, and test mock.
- Rive wrapper using the current recommended React Native runtime and development build. Create a
`ConversationOrb` interface with states: idle, ringing, connecting, voice, videoUnlocked,
videoRequested, video, reconnecting, ended, error. If no final `.riv` asset exists, ship a clean
Reanimated/Skia fallback and an asset contract document.
- Skia components: `TrustRing`, `VoiceWaveform`, `NetworkQualityMiniGraph`, and lightweight
confetti/particle layer. Ensure low-end-device quality levels and reduced-motion alternatives.

Add sample placeholder assets that are legally safe and clearly marked for replacement. Do not
download unlicensed animation files.

Acceptance criteria:
- The app builds when Rive is disabled by feature flag.
- Missing or corrupt animation assets never crash a screen.
- Lottie is not used for live timers or audio-reactive visuals.
- Performance is measured on a low-end Android profile.
```

Prompt 12 - Data, state, forms, mocks, and service adapters

Outcome: Create the frontend data layer that enables all flows without a backend and can later swap to real APIs.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Create the frontend data layer that enables all flows without a backend and can later swap to real
APIs.

IMPLEMENTATION REQUIREMENTS
Implement:
- TanStack Query client with app-focus and online managers for React Native;
- Zustand stores for auth/session, UI preferences, active call, wallet display cache, and development
scenario controls;
- React Hook Form + Zod helpers that map schema errors to accessible field errors;
- `HttpClient` interface and mock/real adapters;
- service interfaces: Auth, Customer, Host, Call, Payment, Wallet, Referral, Earnings, Withdrawal,
Refund, Report, Support, Notification, Analytics;
- deterministic mock database/fixtures with reset and seeded scenarios;
- MSW or an equivalent maintained mock transport for admin web and a mobile-friendly mock adapter;
- latency, failure, empty, offline, and partial-data simulation controls;
- query keys, invalidation policy, optimistic-update rules, and error normalization.
```

```
Build an internal developer scenario screen hidden from production that can switch user role, wallet
balance, host availability, rank, network quality, payment result, report history, and application
status.

Acceptance criteria:
- Feature screens never import fixture files directly.
- Switching from mock to real adapters requires configuration, not screen rewrites.
- Query/store responsibilities are documented and tests cover cache invalidation and error
normalization.
```

Customer mobile experience

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were completed and committed.

Prompt 13 - Customer age gate, login, OTP, and referral entry

Outcome: Implement the complete customer entry flow with polished form behaviour and mock authentication.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the complete customer entry flow with polished form behaviour and mock authentication.

IMPLEMENTATION REQUIREMENTS
Build production-ready screens for splash/bootstrap, mandatory update, 18+ confirmation, underage block, mobile login, optional referral-code sheet, OTP verification, suspended account, and blocked account.

Design/alignment requirements:
- centered brand mark with restrained Lottie splash that immediately falls back to static branding;
- 16 px phone gutters, clear single-column form rhythm, bottom-anchored primary action, keyboard-safe layout;
- phone input with +91 default but extensible country model;
- OTP cells that support paste, autofill, backspace movement, resend timer, and accessibility as one coherent field;
- error messages directly below fields; no toast-only validation;
- loading actions preserve button width and prevent duplicate requests.

Mock behaviours:
- configurable OTP success, invalid, expired, too many attempts, suspended, and new/existing user outcomes;
- referral valid, invalid, already used, and pending qualification states.

Motion/haptics:
- subtle logo entrance, form fade/slide, invalid-field shake only when reduced motion is off, selection haptic on referral apply, success haptic on verification.

Acceptance criteria:
- New customer routes to onboarding; existing customer routes to Home.
- Auth state survives restart via SecureStore-compatible abstraction without storing sensitive mock data in plain text.
- Tests cover OTP paste, resend timing, invalid attempts, and route outcomes.
```

Prompt 14 - Customer profile, language, and permission onboarding

Outcome: Implement the complete customer onboarding sequence and permission education.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the complete customer onboarding sequence and permission education.

IMPLEMENTATION REQUIREMENTS
Build:
- basic profile form with display name, date of birth, gender, optional image selection/crop preview;
- primary and secondary language selection with search, selected state, and empty results;
- permission introduction for notifications, microphone, and camera;
- completion and resume-later logic.

Rules:
- age must remain 18+ after DOB validation;
- do not request microphone/camera system permission during onboarding; request just-in-time at call/video use;
- notification permission may be requested after contextual explanation;
- image upload remains a local mock URI with future service interface;
- store onboarding draft safely and clear it on completion.

Visual requirements:
- progress indicator, consistent header, 24 px section spacing, selectable radio cards, clear optional labels;
- language chips must wrap without misalignment;
- long language names and 200% text scale must work.

Motion/haptics:
- progress bar transitions; selected-language chip uses a 160 ms scale/colour transition and light selection haptic.

Acceptance criteria:
- interrupted onboarding resumes at the correct step.
- Permission denial does not block onboarding.
- Form and navigation tests cover validation, resume, and completion.
```

Prompt 15 - Customer Home, ranked host cards, filters, and language fallback

Outcome: Build the main discovery experience exactly around one-tap voice calling.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
```

```
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Build the main discovery experience exactly around one-tap voice calling.

IMPLEMENTATION REQUIREMENTS
Implement the Home route with:
- header containing brand, wallet balance, Add Money action, and notifications;
- concise voice/video price strip;
- preferred-language context and filter action;
- FlashList of ranked online hosts;
- pull to refresh, pagination mock, skeletons, empty, error, and fallback-language banner.

HostCard content:
- avatar, display name, age, primary language, rating, completed calls, rank badge, online/busy state,
“Video after 3 min”, and one prominent Voice Call button;
- overflow actions for Report and Block;
- absolutely no direct-video button and no mandatory host-profile navigation.

Filters:
- language, online only, top rated, rank, and new hosts in a bottom sheet;
- reset/apply, persisted selection, result count preview, and accessibility.

Motion:
- first-load card stagger capped to visible items;
- availability dot breathing animation only while online and disabled in reduced motion;
- button press compression and haptic;
- host reordering uses layout animation without disorienting jumps.

Acceptance criteria:
- Preferred-language hosts appear first; when none are available, other online hosts appear with
explanation.
- Tapping Voice Call performs balance/permission/availability preflight and routes correctly.
- List performance remains smooth with 500 mock hosts.
```

Prompt 16 - Wallet, add money, payment states, and transactions

Outcome: Implement all customer financial UI using direct rupees and frontend-only payment adapters.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
```

```
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement all customer financial UI using direct rupees and frontend-only payment adapters.

IMPLEMENTATION REQUIREMENTS
Build Wallet overview, Add Money, payment-method selection, processing, success, failure, pending,
transaction history, transaction details, and refund details.

Requirements:
- rupee formatting through a shared currency formatter using tabular numerals;
- quick amounts 100/250/500/1000 and validated custom amount;
- UPI, card, and net-banking presentation through a PaymentAdapter interface;
- mock checkout outcomes configured by scenario controls;
- frontend must never credit wallet directly from a checkout callback: move to “verification pending”,
then consume a mocked verified result;
- pending payment polling with safe cancellation;
- immutable transaction rows with status, reference, timestamp, type, linked call, and support action;
- protect against duplicate pay taps and stale completion callbacks.

Visual/motion:
- prominent balance card with gradient, subtle number transition, skeleton balance, success Lottie,
failure Lottie, warning state for pending;
- payment processing must be calm, not a blocking infinite spinner without explanation.

Acceptance criteria:
- Success, failure, user-cancelled, pending, timeout, and duplicate-callback scenarios are testable.
- Returning from payment restores the originating call/add-money context.
- No financial amount becomes negative due to frontend calculations.
```

Prompt 17 - Outgoing voice call, busy, rejected, and no-answer states

Outcome: Implement the complete pre-connect voice-call experience.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
```

```
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the complete pre-connect voice-call experience.

IMPLEMENTATION REQUIREMENTS
Build microphone permission explanation, system-permission handling, insufficient balance, outgoing
ringing, busy, rejected, no-answer, cancelled, and connection-failed screens.

Call preflight order:
1. user/account allowed;
2. host not blocked and host available;
3. minimum wallet balance for three voice minutes;
4. microphone permission;
5. create mock call intent;
6. ring host with timeout;
7. transition through the call state machine.

Design:
- dark call gradient, centered host avatar, name, status, “Video available after 3 connected minutes”,
and large cancel control;
- call action controls stay inside safe areas;
- busy/no-answer screens suggest alternative hosts without charging;
- insufficient-balance route returns to the call intent after successful add money.

Motion/haptics:
- ConversationOrb ringing state, avatar pulse, cancellable ring animation, medium haptic on accepted,
reject haptic on rejection, no haptic loops.

Acceptance criteria:
- Rapid repeated call taps create one call intent.
- Ring timeout and cancellation clean up timers and media resources.
- No charge is displayed for unconnected outcomes.
- Tests cover every preflight branch.
```

Prompt 18 - Active voice call and three-minute trust experience

Outcome: Implement the central active voice-call screen and exact connected-time video unlock.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the central active voice-call screen and exact connected-time video unlock.
```

```
IMPLEMENTATION REQUIREMENTS
Build a call screen driven by the canonical call state machine, not local booleans.

Required UI:
- host name, voice label, network quality, avatar/ConversationOrb, connected timer, wallet, estimated
remaining time;
- Skia TrustRing around the avatar based on exactly 180 connected seconds;
- video control visibly locked with remaining time, then enabled at eligibility;
- mute, speaker, add money, report, and end controls;
- app background/foreground handling and orientation-safe layout.

Timer rules:
- ringing, reconnecting, and disconnected time do not advance connected duration;
- use a monotonic timestamp strategy, not a fragile interval counter;
- restore timer display after app backgrounding;
- eligibility is emitted once and is testable with an injected clock.

Visualisation:
- waveform uses simulated/mock amplitude through Skia; later replace through CallAdapter audio levels;
- trust ring changes through calm stages, not gamified pressure;
- at 180 seconds use a soft double haptic and non-blocking Lottie sparkle, then reveal Request Video.

Acceptance criteria:
- Timer accuracy tests cover backgrounding, reconnection, clock drift, and unmount.
- End Call requires a deliberate large destructive control and optional confirmation based on
duration.
- Reduced-motion mode replaces pulses with static status changes.
```

Prompt 19 - Video request, camera permission, and seamless media transition

Outcome: Implement the voice-to-video upgrade flow without direct video entry.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the voice-to-video upgrade flow without direct video entry.

IMPLEMENTATION REQUIREMENTS
Build video-request waiting, cancel request, host rejection with cooldown, camera permission, 3-2-1
transition, transition failure, and accepted video states.

Rules:
- Request button exists only after three connected voice minutes.
- Tapping Request Video is the customer confirmation; show the video price before/while waiting but do
```

```
not add another confirm modal.
- Host acceptance is required.
- A rejected request continues voice and starts a configurable cooldown.
- If camera permission is denied, continue voice and explain how to open settings.
- Video billing begins only after the mocked CallAdapter emits remote-video-connected.

Motion:
- same-screen evolution: avatar enlarges, background darkens, controls reposition with Reanimated
layout transitions, self-preview fades in, remote video surface expands;
- countdown remains skippable/cancellable until connection begins;
- failure smoothly returns to voice and voice pricing.

Acceptance criteria:
- No route allows direct video from Home.
- Repeated requests cannot bypass cooldown.
- Voice remains stable during permission and request states.
- State-machine and UI tests cover accept, reject, cancel, permission denial, and video connection
failure.
```

Prompt 20 - Active video call, return to voice, and draggable self preview

Outcome: Implement the production-quality active video-call screen.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the production-quality active video-call screen.

IMPLEMENTATION REQUIREMENTS
Build:
- remote video surface placeholder through CallAdapter;
- draggable self-preview constrained to safe regions and persisted within the call;
- mute, camera on/off, switch camera, speaker, return to voice, add money, report, and end controls;
- auto-hiding controls with accessible always-visible mode;
- network degradation indicators and fallback to audio;
- privacy placeholder when either camera is off.

Gesture/motion:
- Gesture Handler + Reanimated drag/snap for self preview;
- controls fade with touch activity;
- Return to Voice confirmation sheet explains pricing change;
- layout supports portrait and landscape;
- avoid heavy blur over active video on low-end devices.

Billing display:
- show total call timer, video duration, wallet, estimated time, and current video price;
```

```
- if remote video disconnects beyond grace, stop mocked video billing and return to voice state.

Acceptance criteria:
- Self preview never covers End Call or system safe areas.
- Camera off and app background events are handled safely.
- Tests cover return to voice, remote video loss, control auto-hide, and drag boundaries.
```

Prompt 21 - Low balance, add money during call, reconnecting, and report safety

Outcome: Implement all critical in-call interruption and safety states.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement all critical in-call interruption and safety states.

IMPLEMENTATION REQUIREMENTS
Build:
- low-balance banner and modal with estimated time;
- add-money-during-call sheet with quick amounts and configurable grace timer;
- payment success/failure/pending return to the active call;
- reconnecting screen/overlay with 30-second mock window and paused billing display;
- report-during-call sheet with categories, comments, submit, submit-and-end, and preserved draft on
failure;
- block-after-call option;
- automatic end when balance reaches zero in the mock state machine.

Rules:
- warnings must not cover End Call;
- report action is always reachable in voice and video;
- disconnected time is not counted;
- payment UI cannot expose full sensitive payment data;
- no repeated warning haptics every render.

Motion/haptics:
- amber warning pulse once, reconnect orb state, countdown number transition, error haptic only once
per event.

Acceptance criteria:
- Simulate low balance at any call state.
- Reconnect success returns to the previous media mode; failure finalizes call.
- Report submission is idempotent and retains local evidence metadata until accepted by adapter.
- Integration tests cover call + payment + reconnect interactions.
```

Prompt 22 - Call finalization, summary, rating, history, and refund request

Outcome: Implement the complete post-call customer experience.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the complete post-call customer experience.

IMPLEMENTATION REQUIREMENTS
Build Call Ended processing, Call Summary, rating, call history, call details, report issue, and refund request screens.

Summary must show:
- call ID, host, date/time, voice duration, video duration, price snapshots, charges, total, end reason, wallet balance, and refund/verification state;
- pending calculation state when finalization is not ready;
- clear distinction between estimated and finalized amounts.

Rating:
- 1-5 stars, predefined feedback tags, optional comment, skip, and Report Instead;
- only connected calls can be rated once;
- accessible star selection and meaningful error feedback.

History:
- FlashList with all/voice+video/voice/failed/refunded filters;
- pagination mock, pull-to-refresh, skeleton, empty, and error states.

Refund request:
- reason, comments, optional local screenshot metadata, eligibility result, submission status, and linked support ticket.

Acceptance criteria:
- Finalization handles delayed billing and survives app restart.
- Rating does not block returning Home.
- Tests verify duration/amount formatting, one-rating rule, filters, and refund eligibility UI.
```

Prompt 23 - Customer refer-and-earn journey

Outcome: Implement the customer referral experience from code entry through reward history.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
```

```
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the customer referral experience from code entry through reward history.

IMPLEMENTATION REQUIREMENTS
Build Refer & Earn, share sheet, referral rules, referral history, referral detail, reward received,
pending qualification, rejected/fraud-review, and empty states.

Requirements:
- display referral code and shareable deep link;
- native share integration with copy fallback;
- qualification timeline: joined -> minimum add-money met -> first paid call completed -> verification
-> reward released;
- separate new-customer reward and referrer reward copy;
- wallet reward transaction link;
- never promise instant reward before verification;
- frontend anti-abuse messaging without pretending to make fraud decisions.

Motion:
- code copy micro-interaction, progress timeline transitions, one-time Lottie reward celebration,
success haptic.

Acceptance criteria:
- Deep link pre-fills referral code for a new user.
- Existing user cannot retroactively apply a code.
- History supports pending, qualified, rewarded, rejected, and expired states.
- Share and deep-link tests are included.
```

Prompt 24 - Customer profile, privacy, notifications, support, and account lifecycle

Outcome: Complete the remaining customer account and support experience.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
```

```
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Complete the remaining customer account and support experience.

IMPLEMENTATION REQUIREMENTS
Build:
- profile overview/edit, language settings, notification centre, notification preferences;
- privacy and safety, blocked users, report history;
- support ticket list/detail/new ticket with attachments metadata and retry;
- Terms, Privacy Policy, Community Guidelines content shells with version/date;
- logout confirmation, delete-account request, account deletion pending, and deletion blocked by
unresolved mock balance/refund state.

Notifications:
- payment, refund, call, safety, referral, and support categories;
- mark read, mark all read, deep-link navigation, empty/loading/error;
- local notification mocks and Expo notification routing boundary.

Acceptance criteria:
- Sensitive account actions require confirmation and clear consequences.
- Support drafts survive temporary failure.
- Blocked hosts disappear from Home fixtures.
- All legal pages are scrollable, readable, and versioned.
```

Host mobile experience

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were completed and committed.

Prompt 25 - Host login and complete signup profile

Outcome: Implement the host entry and signup flow from mobile number to call-rule acknowledgement.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the host entry and signup flow from mobile number to call-rule acknowledgement.

IMPLEMENTATION REQUIREMENTS
Build host login, OTP, optional referral code, legal/basic information, public display profile, language selection, public photo capture/gallery/crop preview, and voice-first call rules acknowledgement.

Fields:
- legal name separate from public display name;
- DOB with 18+ validation, city/state, languages, profile photo;
- explain that every call starts as voice, video unlocks after three connected minutes, and host can accept/reject.

Design:
- progress stepper, save draft, resume onboarding, validation summary, privacy callouts distinguishing public/private data;
- consistent bottom action alignment and keyboard safety.

Motion/haptics:
- progress transitions, selected language haptic, photo upload progress, no excessive celebration before approval.

Acceptance criteria:
- Draft resumes at the correct step.
- Legal identity data is not shown in public preview.
- Referral code supports customer/host inviter source in the mock contract.
- Tests cover 18+ validation, draft resume, and public/private data mapping.
```

Prompt 26 - Host verification, payout details, agreement, and application status

Outcome: Implement the full host application and review lifecycle.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the full host application and review lifecycle.

IMPLEMENTATION REQUIREMENTS
Build identity-document selection/upload mock, selfie verification mock, payout method (UPI or bank), verification results, Host Agreement, submit confirmation, pending review timeline, approved, rejected, resubmission, suspended, and appeal/support states.

Rules:
- files remain local metadata in frontend mocks; never claim real KYC verification;
- payout fields mask sensitive values after save;
- skip payout only if product rules allow browsing draft, but block withdrawal until verified;
- agreement records version and accepted timestamp;
- rejection identifies exact sections to correct;
- approval assigns starting Bronze rank and payout snapshot.

Motion:
- upload progress, status timeline layout animation, approval Lottie, rejection warning, success/reject haptics.

Acceptance criteria:
- Application state survives restart.
- Approved host reaches dashboard; pending/rejected host cannot go online.
- Resubmission edits only required sections while showing complete summary.
- Tests cover every application transition.
```

Prompt 27 - Host dashboard and availability controls

Outcome: Implement the host operational dashboard with rank and readiness.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
```

```
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the host operational dashboard with rank and readiness.

IMPLEMENTATION REQUIREMENTS
Build Dashboard containing:
- online/offline toggle with confirmation;
- readiness blockers: verification, microphone, notifications, payout warnings;
- current rank, leaderboard position, payout percentage, next-rank progress;
- today earnings, calls, rating, acceptance rate, alerts;
- quick links to leaderboard, earnings, referrals, calls, and profile.

Rules:
- host defaults offline after unsafe errors or invalid session;
- going online triggers permission/readiness checks;
- missed/rejected call impact is explained without manipulative warnings;
- busy state is controlled by active call state.

Motion/haptics:
- availability switch uses semantic toggle haptic, online surface gets a subtle breathing accent,
metric values animate, rank progress uses Reanimated/Skia.

Acceptance criteria:
- Pending/suspended hosts cannot go online.
- Offline state prevents incoming mock calls.
- Dashboard handles no earnings, first-call state, and partial metric failure.
- Tests cover readiness and availability transitions.
```

Prompt 28 - Incoming calls and host active-call controls

Outcome: Implement host-side incoming, voice, video-request, video, reconnect, and end flows.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
```

```
Implement host-side incoming, voice, video-request, video, reconnect, and end flows.

IMPLEMENTATION REQUIREMENTS
Build:
- incoming voice call with customer display info, accept, reject, block, report, and timeout;
- active voice call with customer avatar, connected duration, current estimated earnings, video
eligibility status, mute, speaker, report, block later, end;
- incoming video request showing current video price/payout context and Accept/Reject;
- camera permission and voice-to-video transition;
- active video controls matching customer capabilities;
- reconnecting, safety report, and host call summary.

Rules:
- incoming call full-screen behaviour is represented through a frontend boundary and
notification/deep-link mock;
- host accepts video only after eligibility event;
- earnings pause while disconnected and use rank/pricing snapshots;
- end/report actions are always reachable.

Motion/haptics:
- incoming call Rive/ringing state, accept medium confirm, reject haptic, video request attention
pulse once, seamless media transition.

Acceptance criteria:
- Call state is shared with customer simulation for deterministic dual-role demos.
- Timeout/reject cleans up the request.
- Tests cover accept/reject, video accept/reject, reconnect, and end reasons.
```

Prompt 29 - Host call history, call details, and customer safety

Outcome: Implement host history and post-call actions.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement host history and post-call actions.

IMPLEMENTATION REQUIREMENTS
Build FlashList call history with filters, call details, earning breakdown, pending/final states,
customer report, block customer, support link, and report history.

Details must show:
- call ID, customer display name, voice/video durations, customer charge, platform share, host payout
rate at call time, host earning, settlement state, refund adjustment if later applied;
- no customer phone number or private payment data.
```

```
Safety:
- report reasons, optional comments, linked call, submission progress, block confirmation;
- blocked customer is excluded from incoming mock matching.

Acceptance criteria:
- Rank changes after the call do not rewrite historical payout snapshots.
- Empty and first-call states are polished.
- Tests cover filters, masked data, reports, and blocked matching.
```

Prompt 30 - Host leaderboard, rank details, and rank rules

Outcome: Implement the complete host ranking and leaderboard UX.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the complete host ranking and leaderboard UX.

IMPLEMENTATION REQUIREMENTS
Build:
- weekly/monthly/all-time leaderboard with language filter;
- pinned “You” row when outside the visible top group;
- rank details with score breakdown, position, payout, progress, and next-rank requirements;
- ranking rules and rank history;
- Bronze, Silver, Gold, Platinum, Diamond visual variants.

Requirements:
- rank is based on rating, completed calls, acceptance, returning customers, availability,
cancellation, and complaints;
- do not rank by appearance;
- show data freshness timestamp and calculation period;
- explain that displayed income percentage applies prospectively;
- include loading, insufficient-data/new-host, error, and rank-under-review states.

Visualization:
- Skia progress ring, restrained rank shimmer, no continuous confetti;
- accessible non-colour rank labels.

Acceptance criteria:
- Leaderboard handles 1,000 mock hosts smoothly.
- Rank calculations are display-only from adapter data, never recomputed inconsistently in screens.
- Tests cover pinned position, filter changes, and insufficient data.
```

Prompt 31 - Host earnings, settlements, and withdrawal journey

Outcome: Implement all host financial screens and state transitions.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement all host financial screens and state transitions.

IMPLEMENTATION REQUIREMENTS
Build earnings dashboard, earnings details, pending earnings, available earnings, buffer explanation, request withdrawal, confirmation, submitted, history, details, failed/retry, payout method management, and payout hold states.

Requirements:
- show rank and payout percentage;
- voice/video earnings, gross customer spend, platform share, referral reward, refund adjustment, penalty/hold rows;
- fixed/percentage buffer explanation and maximum withdrawable amount;
- minimum withdrawal and daily limit validation;
- UPI/bank masked details;
- withdrawal state machine: draft -> submitted -> review -> approved -> processing -> completed/failed/rejected/held;
- front-end does not mark payment complete based on optimistic UI.

Motion:
- number transitions, success Lottie, warning states, no casino-like effects.

Acceptance criteria:
- Amount validation uses decimal-safe money utilities.
- Withdrawal cannot violate buffer or available balance.
- Tests cover every status and rank/payout display.
```

Prompt 32 - Host refer-and-earn for customers and hosts

Outcome: Implement both host referral programmes.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
```

```
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement both host referral programmes.

IMPLEMENTATION REQUIREMENTS
Build Refer & Earn overview, refer customer, refer host, native share, code copy, rules, referral
history, referral detail, customer qualification, host approval/valid-call progress, pending reward,
released reward, rejected/fraud-review, and disabled referral access.

Requirements:
- customer referral reward after first qualifying paid call;
- host referral reward after approval and configured valid-call count;
- distinct links/copy for customer and host referral targets;
- progress indicators such as 7/10 valid calls;
- rewards enter pending before release;
- link released rewards to earnings statement.

Motion/haptics:
- progress updates animate; copy/share selection haptic; one-time reward Lottie.

Acceptance criteria:
- Referral types cannot be confused in links or history.
- Disabled/referral-review states prevent sharing but preserve history.
- Tests cover deep links, progress, and reward release display.
```

Prompt 33 - Host profile, verification, support, and account lifecycle

Outcome: Complete all host settings and support screens.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.
```

```
STAGE OBJECTIVE
Complete all host settings and support screens.

IMPLEMENTATION REQUIREMENTS
Build profile overview, public preview, edit public profile, languages, verification status, payout
methods, notification settings, privacy and safety, blocked customers, Host Guidelines, support
tickets, logout, deactivate account, and deactivation blocked by active calls/pending
withdrawals/disputes.

Requirements:
- display rank, payout, rating, verification summary;
- edits requiring moderation show pending state without immediately replacing approved public data;
- payout method changes require re-verification mock state;
- support can link call, withdrawal, referral, or application;
- preserve pending earnings explanation on deactivation.

Acceptance criteria:
- Public preview never leaks legal/KYC/payout fields.
- Host cannot deactivate during active call.
- Tests cover moderation-pending edits and deactivation blockers.
```

Prompt 34 - Rank-change, earnings, and operational notification experiences

Outcome: Implement all host celebration and warning states responsibly.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement all host celebration and warning states responsibly.

IMPLEMENTATION REQUIREMENTS
Build rank increased, rank decreased, payout changed, earnings released, withdrawal completed/failed,
application update, safety warning, referral reward, and availability reminder notification
destinations.

Rank-up experience:
- show old rank -> new rank, old payout -> new payout, effective date, next actions;
- use a single 1-1.5 second Lottie/Skia celebration and success haptic;
- allow immediate skip and respect reduced motion.

Rank-down experience:
- calm explanation, contributing metrics, effective date, Improve My Rank and Contact Support;
- no shame language or aggressive animation.
```

```
Acceptance criteria:
- Notifications deep-link to the correct detail screen.
- Each one-time celebration is marked seen and does not replay every launch.
- Tests cover seen state, deep links, and reduced motion.
```

Admin website

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were completed and committed.

Prompt 35 - Admin app foundation, authentication shell, and six-module layout

Outcome: Build the production admin frontend foundation using only Dashboard, Hosts, Payments, Refunds, Reports, and Settings.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Build the production admin frontend foundation using only Dashboard, Hosts, Payments, Refunds, Reports, and Settings.

IMPLEMENTATION REQUIREMENTS
In `apps/admin`, configure current stable Next.js App Router, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, TanStack Table, React Hook Form, Zod, Lucide, Recharts, and Sentry boundary.

Build:
- login, two-factor mock, forgot-password shell, unauthorized, session-expired;
- responsive sidebar with exactly six primary modules;
- top bar with search placeholder, notifications, environment badge, and admin menu;
- desktop-first content shell that remains usable on tablet widths;
- breadcrumb, page header, filter bar, data table, pagination, detail tabs, modal, drawer, toast, confirmation pattern;
- role/permission frontend guard model stored in mocks, while clearly documenting that real authorization must be server-enforced.

Design:
- shared semantic colours with mobile brand, professional light surfaces, 12-column desktop grid, 24 px page spacing, compact tables, visible keyboard focus.

Acceptance criteria:
- No removed legacy sidebar modules remain.
- Every primary module has a route, loading state, and empty state.
- Keyboard-only navigation can reach all shell controls.
- Production build succeeds.
```

Prompt 36 - Admin Dashboard module

Outcome: Implement the operations dashboard with realistic frontend analytics fixtures.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
```

```
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the operations dashboard with realistic frontend analytics fixtures.

IMPLEMENTATION REQUIREMENTS
Build KPI cards and charts for:
- customers, approved hosts, online hosts, calls, connected minutes, calls reaching three minutes,
video request and acceptance rates, customer payments, host payout, platform income, referral rewards,
pending refunds, open reports, rank distribution, revenue by language, referral conversion.

Requirements:
- date presets and custom range;
- comparison to previous period;
- chart skeletons, no-data, partial failure, and refresh;
- accessible chart summaries and data-table alternative;
- drill-down links only into the six modules;
- export CSV generated from current mock dataset;
- avoid implying data is live when mock mode is enabled.

Acceptance criteria:
- Dashboard works at 1280, 1440, and tablet landscape widths.
- Charts use consistent number/currency/percent formatting.
- Each card has a deterministic query key and independent error recovery.
- Tests cover date filters and exports.
```

Prompt 37 - Hosts module including applications, ranks, earnings, withdrawals, and referrals

Outcome: Implement the entire Hosts operational area inside one primary module.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
```

```
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the entire Hosts operational area inside one primary module.

IMPLEMENTATION REQUIREMENTS
Build subroutes/tabs for:
- host list with status, language, rank, rating, earnings, reports;
- application list/detail, secure-document placeholders, approve, reject, request resubmission;
- host detail tabs: profile, verification, calls, earnings, withdrawals, rank, referrals, reports,
notes;
- force offline, suspend, ban, hold/release earnings with confirmation reason;
- rank management, score breakdown, history, manual adjustment, recalculation mock;
- leaderboard management and export;
- host payout/withdrawal review embedded within host details;
- referral progress and reward holds.

Requirements:
- sensitive actions require reason, confirmation, permission check, and audit-event mock;
- documents use safe placeholders and never expose real identity data in fixtures;
- tables support search, filters, sorting, pagination, column visibility, and URL-synced state.

Acceptance criteria:
- Applications are not a seventh sidebar module.
- Historical call payout remains immutable when rank changes.
- Tests cover filters, application decisions, rank adjustments, and action confirmations.
```

Prompt 38 - Payments module including wallet, earnings, payouts, referrals, and reconciliation

Outcome: Implement the complete Payments operational area.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement the complete Payments operational area.
```

```
IMPLEMENTATION REQUIREMENTS
Build:
- overview KPIs for received, wallet credits, host earnings, payouts, and referral rewards;
- tabs for customer payments, wallet transactions, host earnings, host payouts, referral rewards,
reconciliation;
- payment detail timeline with provider reference, frontend status, wallet-credit status, linked
refund;
- payout detail with balance, buffer, verification, risk flags, approve/reject/hold/complete mock
actions;
- referral reward release/hold actions;
- reconciliation UI comparing app and provider mock statuses;
- CSV exports and audit-event display.

Rules:
- never show full card/bank data;
- UI distinguishes requested, pending verification, verified, completed, failed, cancelled, reversed;
- manual credit/debit actions require reason and prominent warning that server enforcement is required
later.

Acceptance criteria:
- Decimal-safe money formatting is consistent.
- Status changes update related lists through query invalidation.
- Tests cover reconciliation and payout decisions.
```

Prompt 39 - Refunds and Reports modules

Outcome: Implement refund operations and safety/support investigation workflows.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement refund operations and safety/support investigation workflows.

IMPLEMENTATION REQUIREMENTS
Refunds:
- list/detail, linked call/payment, reason, customer history, technical evidence placeholders, system
recommendation, full/partial/reject/request info;
- partial refund allocation between host and platform display;
- processing/pending/completed/failed states.

Reports:
- list/detail for customer reports, host reports, call safety, payment complaint, referral fraud,
ranking abuse, and support tickets;
- priority, assignment, timeline, linked records, evidence metadata, internal notes;
- actions: warn, restrict, suspend, ban, hold earnings, adjust/recalculate rank, close/reopen;
```

```
- dedicated referral-fraud and ranking-abuse review layouts.

Requirements:
- all destructive actions require reason and confirmation;
- provide safe mock evidence, not actual private recordings;
- keep report and refund histories immutable in UI mocks.

Acceptance criteria:
- Refunds and Reports remain separate primary modules.
- Support tickets are handled as a Reports subtype, not a seventh sidebar module.
- Tests cover partial refunds, report actions, assignments, and audit events.
```

Prompt 40 - Settings module and configuration governance

Outcome: Implement all platform configuration screens inside Settings.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement all platform configuration screens inside Settings.

IMPLEMENTATION REQUIREMENTS
Build settings home and sections for:
- call: three-minute unlock, request cooldown, ring timeout, reconnect window, low-balance warning,
max duration;
- pricing: voice/video rates, minimum balances, per-second billing display;
- ranking: metric weights, tier thresholds, payout percentages, recalculation cadence, preview;
- referrals: rewards, qualification conditions, settlement delay;
- withdrawals: minimum, fixed/percentage buffer, settlement, daily limit;
- languages and fallback order;
- safety thresholds and automation display;
- payment provider public configuration/status placeholders;
- notification templates/preferences;
- admin access and audit logs as Settings subroutes.

Requirements:
- Zod validation, dirty-state warning, change preview, effective date, confirmation, and audit record;
- ranking weights validation and preview impact table;
- configuration version history mock;
- make clear that final enforcement is backend-owned later.

Acceptance criteria:
- Invalid or conflicting settings cannot be submitted.
- Preview and reset are available.
- Tests cover schema validation, dirty navigation, and rank preview.
```

Cross-cutting production quality

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were completed and committed.

Prompt 41 - Accessibility and reduced-motion audit

Outcome: Make the entire mobile and admin frontend accessible and motion-safe.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Make the entire mobile and admin frontend accessible and motion-safe.

IMPLEMENTATION REQUIREMENTS
Audit and fix every route and reusable component for:
- semantic roles, labels, hints, state announcements, focus order, heading hierarchy;
- minimum touch targets, keyboard navigation on admin, screen reader navigation on mobile;
- dynamic type and text zoom, long content, colour contrast, colour-independent statuses;
- focus management after navigation, modal open/close, error submission, and async completion;
- reduced motion and disabled haptic preferences;
- call controls and timers with meaningful spoken labels;
- accessible charts with summaries/tables;
- form errors associated with fields.

Add automated accessibility checks where supported and a manual QA checklist for VoiceOver, TalkBack, keyboard, and 200% text scale.

Acceptance criteria:
- No icon-only action lacks an accessible name.
- Decorative animation is hidden from screen readers.
- Core auth, payment, call, withdrawal, refund, and destructive admin actions pass manual checklist.
```

Prompt 42 - Localization and formatting readiness

Outcome: Internationalize user-facing copy and locale-sensitive formatting without changing the preferred-language matching logic.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.
```

```
Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Internationalize user-facing copy and locale-sensitive formatting without changing the preferred-
language matching logic.

IMPLEMENTATION REQUIREMENTS
Implement an i18n layer for mobile and admin with English as source and starter Telugu/Hindi resource
structure.

Requirements:
- no user-facing strings buried in feature logic;
- pluralisation, interpolation, dates, times, durations, rupees, percentages, rank, phone masking;
- locale-aware but consistent Indian currency display;
- support long translations, line wrapping, and RTL architecture readiness;
- preferred conversation language remains product data, separate from UI language;
- legal content versioning per locale;
- missing-key behaviour visible in development and safe in production.

Add translation extraction/check script and tests for key formatting cases.

Acceptance criteria:
- Switching UI language does not alter host-matching preference.
- Timers remain stable and numerically readable.
- No raw concatenated sentence fragments prevent translation.
```

Prompt 43 - Offline, network quality, and resilience

Outcome: Implement robust offline and degraded-network behaviour across both frontends.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
```

```
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Implement robust offline and degraded-network behaviour across both frontends.

IMPLEMENTATION REQUIREMENTS
Mobile:
- integrate network status with TanStack Query online manager;
- offline banner, cached last-safe data, retry queue for nonfinancial drafts, reconnect behaviour;
- never queue financial completion or call-accept actions blindly;
- network quality levels for call UI through adapter events;
- app-state and background handling.

Admin:
- offline banner, stale-data labels, retry, preserved filters/forms, no fake success on mutations.

Create centralized error normalization and user-safe messages for timeout, offline, permission,
validation, provider unavailable, unauthorized, rate limited, and unknown errors.

Acceptance criteria:
- Mock offline mode can be toggled from developer scenarios.
- Draft reports/support forms survive connection loss.
- Financial and destructive actions clearly remain pending or failed rather than optimistic success.
```

Prompt 44 - Frontend security and privacy hardening

Outcome: Harden the frontend against common data leakage and unsafe client assumptions.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Harden the frontend against common data leakage and unsafe client assumptions.

IMPLEMENTATION REQUIREMENTS
Implement and document:
- SecureStore abstraction for tokens/session placeholders; MMKV or equivalent only for nonsensitive
preferences/cache;
- redaction of mobile numbers, bank/UPI, identity documents, and provider references in logs/screens;
- no secrets in app config, JavaScript bundle, source maps, or repository;
- screenshot/screen-capture deterrence boundary on sensitive verification/video screens where
supported, with clear limitations;
- safe deep-link allowlist and parameter validation;
- URL and file upload metadata validation;
- prevention of open redirects and untrusted HTML rendering;
- clipboard timeout for copied referral/payment references where appropriate;
```

```
- production removal of developer scenario tools;
- logout/session-expiry cleanup of sensitive state and query cache.

Add `docs/frontend-security.md` separating frontend controls from mandatory backend enforcement.

Acceptance criteria:
- Secret scanning passes.
- Production logs do not include PII.
- Invalid deep links and malformed adapter payloads fail safely.
```

Prompt 45 - Analytics, observability, and Sentry integration

Outcome: Add privacy-conscious frontend analytics and production error monitoring.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Add privacy-conscious frontend analytics and production error monitoring.

IMPLEMENTATION REQUIREMENTS
Implement an AnalyticsService interface with mock console adapter in development and disabled-by-
default production provider placeholder.

Define events for:
- onboarding steps, host list viewed, call initiated/connected/end, three-minute reached, video
requested/accepted/rejected, add-money funnel, referral funnel, host availability, rank
viewed/changed, withdrawal funnel, admin operational actions.

Rules:
- no raw phone number, legal name, bank/UPI, document, free-text report, or video/audio data in
events;
- stable event names and typed properties;
- route tracking and correlation IDs that do not expose PII.

Integrate Sentry for mobile and admin with environment/release tags, source maps, error boundaries,
navigation tracing, and conservative production sampling. Add scrub-before-send hooks.

Acceptance criteria:
- Deliberate test errors appear with correct release/environment in a nonproduction project.
- PII scrub tests pass.
- Analytics can be disabled by user/privacy configuration without breaking app flow.
```

Prompt 46 - Performance, memory, and bundle optimization

Outcome: Profile and optimize the complete frontend for low-end Android and production web.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Profile and optimize the complete frontend for low-end Android and production web.

IMPLEMENTATION REQUIREMENTS
Mobile tasks:
- measure Home FlashList, leaderboard, transaction history, active call, Rive/Skia/Lottie screens;
- remove unnecessary rerenders using selectors and memoization only where measured;
- optimise image sizing/caching, animation asset size, explicit Lucide imports, route lazy loading, query cache limits;
- implement animation quality tiers and disable expensive particles on low-end devices;
- ensure timers and call state do not rerender whole screens every second;
- inspect memory cleanup for media placeholders, gestures, listeners, timers, and animations.

Admin tasks:
- route splitting, table virtualization/pagination, chart lazy loading, package import optimisation, image/font optimisation, Web Vitals.

Create `docs/performance-budget.md` with budgets for startup, list responsiveness, frame rate, JS bundle/assets, admin page load, and regression procedure.

Acceptance criteria:
- No known listener/timer leaks.
- Scrolling and call controls remain responsive under mock stress.
- Production bundles are analysed and documented.
```

Prompt 47 - Unit and component testing completion

Outcome: Create comprehensive automated unit and component coverage.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
```

```
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Create comprehensive automated unit and component coverage.

IMPLEMENTATION REQUIREMENTS
Mobile:
- Jest configured for Expo, React Native Testing Library, Reanimated/Lottie/Rive/Skia mocks, fake
timers, query/store providers;
- test domain state machines, money/duration/formatters, Zod forms, route guards, buttons, fields,
HostCard, TrustRing semantics, payment and call flows, accessibility labels.

Admin:
- Vitest or Jest consistent with Next.js support, Testing Library, mocked Query client,
table/filter/form tests.

Requirements:
- tests assert user-visible behaviour, not implementation details;
- deterministic time and random seeds;
- coverage thresholds targeted at domain/state/service/component layers, not superficial 100%;
- no snapshot-only critical tests;
- add fixture builders.

Acceptance criteria:
- All tests pass from root command.
- Critical state machines have branch coverage.
- Test failures provide actionable messages.
```

Prompt 48 - End-to-end and visual regression testing

Outcome: Automate critical mobile and admin journeys before deployment.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.
```

```
STAGE OBJECTIVE
Automate critical mobile and admin journeys before deployment.

IMPLEMENTATION REQUIREMENTS
Mobile with Maestro:
- customer new-user onboarding;
- add money success/failure;
- Home -> voice -> simulated three-minute unlock -> video request accept/reject -> summary -> rating;
- low balance/reconnect/report;
- host signup/application approved;
- host incoming call, rank, withdrawal, referral.

Admin with Playwright:
- login mock;
- dashboard filters;
- host application approval/rejection;
- payment reconciliation;
- partial refund;
- report safety action;
- settings validation and rank preview.

Add screenshot baselines for core reference screens at defined viewport/device sizes. Mask dynamic
timers and IDs. Integrate tests with CI-friendly mock mode.

Acceptance criteria:
- E2E tests run repeatably from documented commands.
- Flows do not depend on external payment/call/backend providers.
- Visual diffs are reviewed and thresholds documented.
```

Prompt 49 - Brand assets, splash, icons, and animation asset readiness

Outcome: Prepare complete release-grade visual assets without relying on unlicensed files.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Prepare complete release-grade visual assets without relying on unlicensed files.

IMPLEMENTATION REQUIREMENTS
Create:
- app icon, adaptive Android icon layers, iOS icon, favicon/admin metadata, monochrome notification
icon, splash screen, placeholders for store screenshots;
- branded static fallback for every Lottie/Rive asset;
- asset naming, dimensions, compression, ownership/licence manifest;
- call background gradients, rank badge SVGs, empty-state illustrations created in-house or clearly
```

```
licensed;
- preloading strategy for first-call animation assets;
- no remote runtime dependency for critical visuals.

If final brand name/logo is not supplied, use a clearly isolated temporary brand package and
`BRAND_REPLACE.md` listing every replacement location.

Acceptance criteria:
- Icons render without transparent-edge or adaptive mask issues.
- Splash does not flash white between native and React content.
- Production build contains no giant or unused animation assets.
```

Prompt 50 - CI/CD, EAS mobile delivery, OTA updates, and admin deployment

Outcome: Create deployment automation for the frontend monorepo.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing
repository and inspect its current state before editing. Implement the requested stage completely; do
not only explain or provide snippets. This stage is frontend-only: do not create a real backend,
database, payment verification server, OTP service, or Agora token service. Represent external systems
through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider
switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Create deployment automation for the frontend monorepo.

IMPLEMENTATION REQUIREMENTS
Mobile:
- configure EAS development, preview/internal, and production build profiles;
- environment/channel mapping, version/runtime policy, Android package and iOS bundle placeholders;
- EAS Update preview and production channels with safe rollback/republish guidance;
- EAS Submit configuration placeholders and first-time store submission notes;
- source map upload for Sentry;
- production developer tools removed.

Admin:
- production Next.js build, environment validation, Dockerfile or target hosting configuration, health
route, security headers, CSP starter, cache policy, and deployment instructions for Vercel or a Node
container.

CI:
- install with frozen lockfile, format check, lint, typecheck, unit tests, builds, optional E2E,
artifact retention;
- branch/PR preview policy and protected production deployment;
- no secrets in workflow files.

Acceptance criteria:
- Preview mobile binaries can be generated and shared internally.
- Admin production build starts successfully.
- OTA updates are limited to non-native-compatible changes and documented.
```

Prompt 51 - Release QA, store readiness, and operational documentation

Outcome: Run a full frontend release-readiness audit and create operator/developer documentation.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use `npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review, external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Run a full frontend release-readiness audit and create operator/developer documentation.

IMPLEMENTATION REQUIREMENTS
Audit:
- every route, primary/secondary action, back behaviour, loading/empty/error/offline state;
- Android/iOS safe areas, keyboards, permissions, app background, orientation, small/large devices;
- admin responsive widths and keyboard accessibility;
- animation reduced-motion, haptics disabled, missing assets;
- mock-to-real adapter switches, environment variables, production flags;
- PII logging, developer tools, source maps, licences, dependency vulnerabilities;
- test, bundle, and performance budgets.

Create:
- `README.md` quick start;
- `docs/mobile-development.md`, `docs/admin-development.md`, `docs/architecture.md`, `docs/design-system.md`, `docs/motion-haptics.md`, `docs/mock-scenarios.md`, `docs/backend-integration-contracts.md`, `docs/release-runbook.md`, `docs/known-external-prerequisites.md`;
- release checklist for Play Store, App Store, EAS Update, and admin deployment.

Acceptance criteria:
- No unresolved frontend TODO is hidden.
- All genuinely backend-owned items are listed explicitly.
- A new developer can run and review all flows from a clean checkout.
```

Prompt 52 - Final integration, remediation, and deployable frontend sign-off

Outcome: Treat the repository as a release candidate, integrate every previous stage, repair gaps, and produce final sign-off.

```
You are the principal frontend engineer responsible for this production codebase. Work in the existing repository and inspect its current state before editing. Implement the requested stage completely; do not only explain or provide snippets. This stage is frontend-only: do not create a real backend, database, payment verification server, OTP service, or Agora token service. Represent external systems through typed interfaces, mock adapters, deterministic fixtures, and environment-based provider switches. Preserve all previously completed flows.

Global engineering rules:
- Use strict TypeScript with no implicit any and no untyped JSON crossing module boundaries.
```

```
- Prefer current stable, mutually compatible package versions. For Expo-managed native packages use
`npx expo install`; record exact versions in `docs/dependency-matrix.md` and run `npx expo-doctor`.
- Use pnpm, deterministic lockfiles, absolute imports, feature folders, and dependency boundaries.
- Do not leave dead screens, fake buttons, console errors, broken routes, unexplained TODOs, or
placeholder text that looks production-ready.
- Every asynchronous screen must have loading, empty, success, recoverable error, and offline
behaviour where relevant.
- Every interactive control needs disabled, pressed, loading, focus/accessibility, and error states.
- Apply safe-area handling, keyboard avoidance, minimum 44x44 touch targets, semantic accessibility
labels, and reduced-motion support.
- Keep animations purposeful and interruptible. Never block navigation until a decorative animation
finishes.
- Add or update tests for the behaviour implemented in this stage.
- Run format, lint, typecheck, relevant tests, and builds before finishing.
- At completion, report changed files, commands run, test results, screenshots or routes to review,
external prerequisites, and any genuine remaining limitation.

STAGE OBJECTIVE
Treat the repository as a release candidate, integrate every previous stage, repair gaps, and produce
final sign-off.

IMPLEMENTATION REQUIREMENTS
Perform a complete implementation review rather than assuming previous prompts were applied correctly.

1. Compare repository routes and features against `docs/screen-inventory.md` and the approved
customer, host, and six-module admin requirements.
2. Find and fix broken imports, dead actions, inconsistent tokens, unhandled state-machine
transitions, missing states, duplicate components, navigation loops, inaccessible controls, animation
conflicts, and test gaps.
3. Verify there is no direct-video entry from Home; every video call originates from an eligible
connected voice call and host acceptance.
4. Verify referral flows, rank-based payout display, leaderboard, withdrawal buffer, refunds, reports,
and settings are complete.
5. Verify all external capabilities are behind adapters with safe mock implementations and explicit
real-integration contracts.
6. Run root format check, lint, typecheck, unit/component tests, admin E2E, mobile Maestro smoke
flows, Expo Doctor, mobile preview builds where credentials permit, admin production build, dependency
audit, and bundle analysis.
7. Fix all frontend-owned failures. Do not mark the project ready with red checks.
8. Generate `FRONTEND_RELEASE_REPORT.md` containing:
   - exact commit/revision;
   - dependency versions;
   - completed feature matrix;
   - commands and outcomes;
   - device/browser coverage;
   - performance/accessibility/security findings;
   - build/deployment instructions;
   - backend/provider prerequisites that prevent live commercial operation;
   - final go/no-go decision for frontend deployment.

Final acceptance criteria:
- The mobile app and admin website build from a clean checkout.
- Mock mode demonstrates every supported flow without backend access.
- No broken route, dead button, uncaught error, secret, PII log, or release-blocking frontend defect
remains.
- The project is deployable as a frontend preview/production shell, with live OTP, payments, calls,
wallet settlement, refunds, and payouts explicitly waiting for backend/provider integration.
```

Appendix A - Recommended repository shape after completion

```
voice-first-platform/
├── apps/
│   ├── mobile/
│   │   ├── app/                        Expo Router routes
│   │   ├── src/components/             Mobile component system
│   │   ├── src/features/               Auth, customer, host, calls, wallet, etc.
│   │   ├── src/services/               Typed adapters and providers
│   │   ├── src/store/                  Zustand stores
│   │   ├── src/animations/             Lottie/Rive/Skia wrappers
│   │   ├── assets/                     Local release assets
│   │   └── eas.json
│   └── admin/
│       ├── app/                        Next.js App Router
│       ├── components/                 Admin UI patterns
│       ├── features/                   Six operational modules
│       └── public/
├── packages/
│   ├── design-tokens/
│   ├── domain/
│   ├── eslint-config/
│   ├── tsconfig/
│   └── test-utils/
├── docs/
├── .maestro/
├── playwright/
├── pnpm-workspace.yaml
└── package.json
```

Appendix B - Frontend/backend boundary

- Frontend can be fully designed, animated, tested, built, and deployed with mocks.
- Backend is still required for secure OTP, session issuance, real host/customer records, call matching, Agora token generation, wallet ledger, Razorpay order creation and signature verification, pricing enforcement, payouts, refunds, fraud/rank calculation, reports, and admin authorization.
- Never convert a mock adapter into a “temporary production” implementation that trusts frontend balances, billing timers, payment callbacks, rank scores, or admin permissions.

Appendix C - Official documentation baseline

1. Expo project creation: https://docs.expo.dev/get-started/create-a-project/
2. Expo Router: https://docs.expo.dev/router/introduction/
3. Expo development builds: https://docs.expo.dev/develop/development-builds/introduction/
4. React Native setup: https://reactnative.dev/docs/environment-setup
5. Reanimated: https://docs.swmansion.com/react-native-reanimated/
6. Gesture Handler: https://docs.swmansion.com/react-native-gesture-handler/docs/
7. Lottie React Native: https://github.com/lottie-react-native/lottie-react-native
8. Rive React Native runtime: https://rive.app/docs/runtimes/react-native/react-native
9. React Native Skia: https://shopify.github.io/react-native-skia/
10. Gorhom Bottom Sheet: https://gorhom.dev/react-native-bottom-sheet/
11. FlashList: https://shopify.github.io/flash-list/
12. TanStack Query React Native: https://tanstack.com/query/latest/docs/framework/react/react-native
13. NativeWind installation: https://www.nativewind.dev/docs/getting-started/installation

14. Expo Haptics: https://docs.expo.dev/versions/latest/sdk/haptics/
15. Zod: https://zod.dev/
16. Lucide React Native: https://lucide.dev/guide/packages/lucide-react-native
17. Next.js installation: https://nextjs.org/docs/app/getting-started/installation
18. Tailwind with Next.js: https://tailwindcss.com/docs/installation/framework-guides/nextjs
19. shadcn/ui with Next.js: https://ui.shadcn.com/docs/installation/next
20. TanStack Table: https://tanstack.com/table/latest/docs/framework/react/react-table
21. Expo EAS Build: https://docs.expo.dev/build/introduction/
22. Expo EAS Submit: https://docs.expo.dev/submit/introduction/
23. Expo EAS Update: https://docs.expo.dev/eas-update/introduction/
24. Expo unit testing: https://docs.expo.dev/develop/unit-testing/
25. React Native Testing Library: https://callstack.github.io/react-native-testing-library/docs/start/quick-start
26. Maestro: https://docs.maestro.dev/get-started/quickstart
27. Sentry React Native: https://docs.sentry.io/platforms/react-native/
28. Expo Notifications: https://docs.expo.dev/versions/latest/sdk/notifications/

Version note: The prompts intentionally tell the coding agent to resolve and lock current stable compatible versions. Native-library compatibility changes faster than product requirements, so the dependency matrix and Expo Doctor checks are part of the deliverable rather than relying only on version numbers printed in this PDF.